

## PERTEMUAN 10 TEXTURING

### Kemampuan Akhir yang Diharapkan (KAD):

Mahasiswa mampu menerapkan teknik texturing untuk meningkatkan kualitas visual objek.

### Pokok Bahasan:

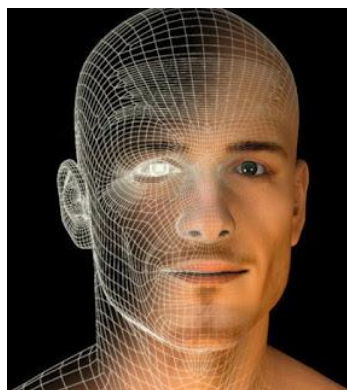
1. Texturing
2. Teknik-teknik realisme dalam komputer grafik (Texture-Maps, Environment-Maps, Bump-Maps, Normal-Maps, Shadow-Maps)
3. Contoh program dengan java processing
4. Soal Latihan

### 10.1 Texturing

*Texturing* adalah proses menambahkan gambar atau pola (disebut texture map) ke permukaan objek 3D untuk memberikan detail visual seperti warna, pola material, atau kesan permukaan tanpa menambah kompleksitas geometrinya. Dalam grafika komputer modern (misalnya di game engine seperti Unity, Unreal, atau Blender), *texturing* adalah tahap penting untuk mencapai realitas visual (visual realism).

Dalam dunia visual, *texturing* adalah proses pemberian karakteristik permukaan pada objek. Maksud dari karakteristik adalah termasuk pewarnaan, kilauan, dan lainnya. Pada umumnya teksturing adalah pemberian warna pada permukaan objek atau pengecatan, walaupun ada proses yang mengubah geometri objek. Dalam software seperti 3DSMax dan Blender, untuk menambahkan tekstur pada objek, kita bisa menggunakan tools Map.

Teknik teksturing adalah termasuk langkah terakhir dalam pendesaian 3D. Hal ini dikarenakan setelah langkah *teksturing* ini langkah selanjutnya hanyalah tinggal melakukan rendering jika ingin menjadi objek 2D. Untuk desain tekstur itu sendiri terdiri dari berbagai macam tipe. Secara default biasanya hanya disediakan tekstur sederhana seperti *wood*, *metal*. Sedangkan untuk tekstur tingkat tinggi seperti tekstur manusia kita bisa mendesainnya sendiri atau mengunduh dari *website*.



Teksturing sangat penting dalam desain 3D atau animasi, karena dengan teksturing inilah objek akan terlihat menjadi hidup. Karena jika hanya dengan warna biasa, suatu objek tidak akan terlihat hidup, namun dengan tekstur kita bisa mengubah gambar biasa menjadi hidup. Dengan adanya tekstur juga bisa mengubah objek yang sebenarnya kurang halus menjadi terlihat lebih halus, jika dibandingkan dengan objek tanpa tekstur.

*Texturing* pada citra yakni frekuensi perubahan rona pada citra yang dinyatakan dengan kasar (*coarseness*), sedang (*regularity*), dan halusness (*smoothness*) suatu permukaan pada citra tersebut. Aspek tekstural dari sebuah citra dapat dimanfaatkan sebagai dasar dari segmentasi, klasifikasi, maupun interpretasi citra. Tekstur dapat didefinisikan sebagai fungsi dari variasi spasial intensitas piksel (nilai keabuan) dalam citra.

Tujuan *Texturing* adalah:

1. Memberi kesan detail permukaan (kayu, logam, kulit, kain, dll).
2. Menghemat sumber daya komputasi (tidak perlu model 3D dengan poligon terlalu banyak).
3. Memberi efek visual tambahan seperti kilau, kedalaman, atau pantulan.

Terdapat tiga masalah utama yang berhubungan dengan tekstur yaitu :

1. Segmentasi Tekstur (*Texture segmentation*) merupakan masalah yang memecah suatu citra ke dalam beberapa komponen dimana tekstur dianggap konstan. Segmentasi tekstur melibatkan representasi suatu tekstur, dan penentuan dasar dimana batas segmen akan ditentukan.
2. Sintesis Tekstur (*Texture synthesis*) yaitu berusaha untuk membangun region tekstur besar yang berasal dari contoh citra kecil yang ada. Dengan menggunakan contoh citra akan dibangun model probabilitas tekstur tersebut, dan kemudian menggambarkannya pada model probabilitas untuk menentukan tekstur citra
3. Bentuk Tekstur (*Shape from Texture*) yaitu melibatkan perbaikan orientasi permukaan atau bentuk permukaan dari tekstur. Di sini diasumsikan bahwa tekstur “kelihatan sama” pada titik-titik yang berbeda pada suatu permukaan, ini artinya bahwa deformasi tekstur dari titik ke titik adalah petunjuk bentuk permukaan.

Berdasarkan strukturnya, tekstur dapat diklasifikasikan dalam 2 golongan yaitu :

1. Makrostruktur; tekstur ini memiliki perulangan pola local secara periodik dalam suatu daerah citra, biasanya terdapat pada pola-pola buatan manusia dan cenderung mudah untuk direpresentasikan secara matematis
2. Mikrostruktur; pada tekstur ini, pola-pola lokal dan perulangan tidak terjadi begitu jelas, sehingga tidak mudah untuk memberikan definisi tekstur yang komprehensif.

Secara garis besar teksturing merupakan salah satu teknik rendering yang pada intinya ingin merubah gambar geometris untuk menghasilkan gambar yang lebih realistis.

## 10.2 Konsep Dasar Texturing

### 1. Texture Map

Texture map adalah gambar dua dimensi yang digunakan untuk menutupi permukaan objek tiga dimensi. File tekstur biasanya berupa format gambar umum seperti .png, .jpg, .bmp, atau .tga.

Contoh:

wood\_texture.jpg → menampilkan pola kayu.

metal\_rust.png → permukaan besi berkarat.

### 2. Texture Coordinates

Setiap titik (vertex) pada model 3D memiliki koordinat tekstur ( $u$ ,  $v$ ) untuk menentukan bagian mana dari gambar yang akan ditempelkan.

| Sistem Koordinat | Deskripsi                              |
|------------------|----------------------------------------|
| $u$              | sumbu horizontal (0 → kiri, 1 → kanan) |

|          |                                      |
|----------|--------------------------------------|
| <b>v</b> | sumbu vertikal (0 → bawah, 1 → atas) |
|----------|--------------------------------------|

Contoh:

Sebuah titik objek memiliki  $(u, v) = (0.5, 0.5) \rightarrow$  berarti mengambil warna dari **tengah** gambar tekstur.

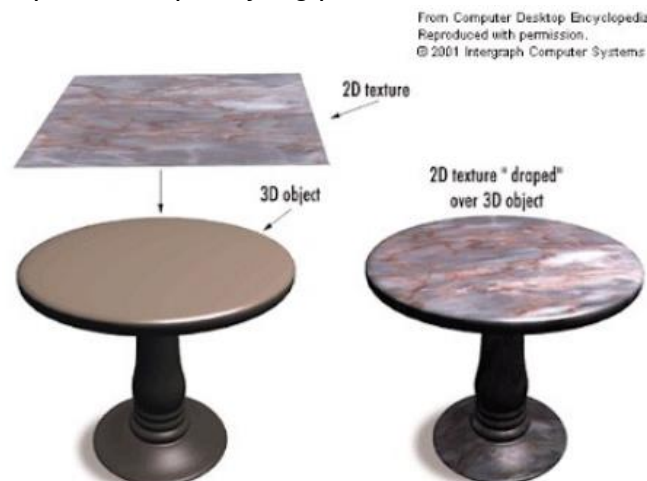
### 3. Proses Texture Mapping

Tahapan:

- Persiapan Gambar Tekstur: Gambar dipilih atau dibuat sesuai jenis permukaan (misal, "batu.jpg" untuk dinding).
- UV Mapping: Permukaan objek 3D "diratakan" ke bidang 2D untuk menentukan koordinat  $(u, v)$ .
- Interpolasi Koordinat Tekstur: Koordinat  $(u, v)$  tiap vertex diinterpolasi di antara piksel (fragment).
- Sampling Warna Tekstur: Sistem mengambil nilai warna dari gambar berdasarkan posisi  $(u, v)$ .
- Rendering: Warna hasil tekstur dikombinasikan dengan pencahayaan (lighting) untuk ditampilkan.

### 10.3 Texture-Maps

Maps berwujud gambar tekstur 2D yang dituangkan ke permukaan geometri/objek untuk membuat penampilan objek tersebut tampak halus untuk detail permukaannya. Pada pengembangan grafik realisme tingkat tinggi diperlukan lebih banyak lapisan tekstur, agar hasil mapping mendekati kesempurnaan. Sebuah tekstur maps yang diterapkan (dipetakan) biasanya dipakai untuk permukaan bentuk objek polygon, proses ini mirip dengan menerapkan texture pada kertas berpola kotak putih yang polos.



Kegiatan texture mapping untuk texture maps dapat didefinisikan sebagai sebuah metode untuk menambahkan detail tekstur permukaan (bitmap atau raster image), atau warna yang dihasilkan komputer grafis atau model 3D. Penerapannya pada grafis 3D dirintis oleh Dr. Edwin Catmull pada proyek Ph.D tesisnya tahun 1974.

### 10.4 Environment-Maps

*Environment-Maps* adalah Maps yang memiliki pewarnaan dengan banyak gambar (tekstur) yang merekam refleksi global dan pencahayaan pada objek. Gambar-gambar ini difungsikan sebagai resampled selama proses rendering, dengan tujuan untuk mengekstrak, dan melihat informasi spesifik yang kemudian diterapkan sebagai tekstur objek pada permukaan geometri.

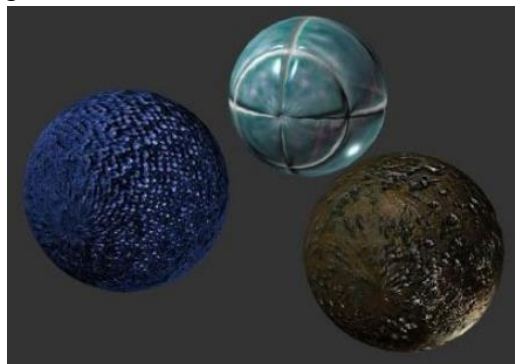
Maps ini sering disebut reflection yang berarti tekstur pencahayaan dari benda-benda luar di sekitar objek, maps jenis ini sangat cocok untuk benda-benda yang memiliki tingkat refleksi yang sangat tinggi, seperti kaca, dan lain sebagainya.



Environment mapping pada maps ini memiliki definisi yaitu metode yang efisien untuk simulasi kompleks permukaan bercahaya melalui suatu gambar tekstur. Tekstur yang digunakan bekerja untuk menyimpan gambar dari lingkungan sekitar objek yang diberikan. Ada beberapa cara untuk menyimpan tekstur lingkungan sekitar objek, salah satu metode yang paling umum adalah metode Spherical Environment Mapping, di mana dalam metode ini suatu tekstur yang berisi citra lingkungan sekitar akan direfleksikan sebagai bola cermin, dan kemudian dimasukkan sebagai tekstur objek yang diberi maps.

### 10.5 Bump-Maps

*Bump maps* adalah maps yang memfokuskan permukaan tekstur pada objek, dengan menggunakan maps ini, permukaan geometri/objek akan terlihat berbeda dan unik, tidak seperti objek-objek yang memiliki permukaan normal pada umumnya. Rumus pekerjaan dari maps ini akan menerapkan bidang untuk intensitas dalam maps yang menunjukkan tingkat rendah tingginya suatu permukaan objek, fungsi maps ini akan mengacaukan permukaan objek yang normal, sehingga menimbulkan kesan unik ketika hasilnya sudah di-render.



Untuk kegiatannya, bump mapping adalah sebuah teknik grafis komputer di mana pada setiap pixel untuk permukaan normal objek, akan diberikan efek tidak halus dan diterapkan sebelum melakukan perhitungan iluminasi. Hasilnya, objek yang diberi maps akan lebih kaya, serta lebih rinci untuk representasi permukaan objek dan lebih mirip pada permukaan alam.

### 10.6 Normal-Maps

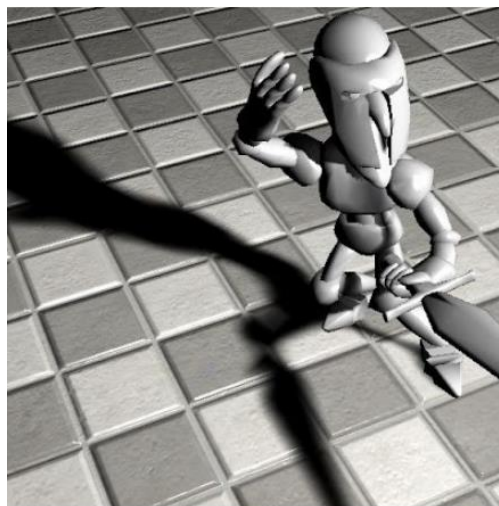
Maps ini serupa dengan bump-maps, perbedaannya adalah fungsionalitas yang ada pada normal maps, maps ini secara alami akan menghasilkan bentuk permukaan yang tidak halus tanpa menggunakan lebih banyak penambahan atau pengurangan poligon pada objek.



Dalam dunia grafik komputer 3D, normal mapping pada maps ini bisa disebut "Dot3 bump mapping", definisinya adalah sebuah teknik yang digunakan untuk "berpura-pura" menambahkan efek tidak halus pada permukaan objek. Kenapa disebut "berpura-pura" ? Hal ini disebabkan karena mapping ini bekerja dengan menambahkan rincian tidak halus pada permukaan objek tanpa menggunakan poligon. Normal mapping biasanya diakurasi sebagai sebuah gambar RGB yang berhubungan dengan koordinat X, Y, dan Z dari permukaan normal suatu objek. Penggunaan umum teknik ini adalah untuk lebih meningkatkan tampilan dan detail dari model poligon rendah dengan menghasilkan peta normal dari model polygon yang tinggi.

### 10.7 Shadow-Maps

*Shadow maps* yaitu maps yang menghasilkan suatu bayangan tekstur pada objek dengan menangkap siluet objek tersebut dari sumber cahaya yang terlihat. Maps ini sangat sering dipakai oleh pengembang grafik 3D, karena efek yang dihasilkan seolaholah menunjukkan objek tersebut sangat realistis, dan disebabkan adanya bayangan dari sumber cahaya yang ada.



Fungsi kerjanya yang berkonsep shadow mapping adalah proses di mana bayang-bayang ditambahkan ke grafik komputer 3D. Konsep ini diperkenalkan oleh Lance Williams pada tahun 1978, dalam sebuah makalah yang berjudul "*Casting curved shadows on curved surfaces*". Sejak saat itu, teknik ini telah digunakan baik dalam pra-render adegan, dipakai secara realtime, atau bahkan diterapkan ke banyak konsol dan PC *high-end game*.

## 10.8 Contoh Program

### 1. Kubus dengan tekstur

Textur image: <https://gabrielgambetta.com/computer-graphics-from-scratch/images/crate-texture.jpg>

```
PImage tex;
float rotX = 0, rotY = 0;

void setup() {
  size(800, 600, P3D);
  tex = loadImage("wood.jpg"); // pastikan file ada di folder "data"
  noStroke();
}

void draw() {
  background(30);
  lights(); // aktifkan pencahayaan
  translate(width/2, height/2, 0); // pusatkan kubus
  rotateX(rotX);
  rotateY(rotY);
  textureMode(NORMAL);
  beginShape(QUADS);
  texture(tex);

  float s = 100; // setengah sisi kubus

  // --- DEPAN ---
  vertex(-s, -s, s, 0, 0);
  vertex(s, -s, s, 1, 0);
  vertex(s, s, s, 1, 1);
  vertex(-s, s, s, 0, 1);

  // --- BELAKANG ---
  vertex(s, -s, -s, 0, 0);
  vertex(-s, -s, -s, 1, 0);
  vertex(-s, s, -s, 1, 1);
  vertex(s, s, -s, 0, 1);

  // --- KANAN ---
  vertex(s, -s, s, 0, 0);
  vertex(s, -s, -s, 1, 0);
  vertex(s, s, -s, 1, 1);
  vertex(s, s, s, 0, 1);

  // --- KIRI ---
  vertex(-s, -s, -s, 0, 0);
  vertex(-s, -s, s, 1, 0);
  vertex(-s, s, s, 1, 1);
  vertex(-s, s, -s, 0, 1);

  // --- ATAS ---
  vertex(-s, -s, -s, 0, 0);
  vertex(s, -s, -s, 1, 0);
  vertex(s, -s, s, 1, 1);
  vertex(-s, -s, s, 0, 1);

  // --- BAWAH ---
  vertex(-s, s, s, 0, 0);
  vertex(s, s, s, 1, 0);
  vertex(s, s, -s, 1, 1);
  vertex(-s, s, -s, 0, 1);

  endShape();

  rotX += 0.01;
  rotY += 0.02;
}
```

## 2. Kubus dengan tekstur berbeda setiap sisi

```

PImage[] tex = new PImage[6];
float rotX = 0, rotY = 0;

void setup() {
  size(800, 600, P3D);

  // Load keenam gambar tekstur
  tex[0] = loadImage("img0.jpg"); // depan
  tex[1] = loadImage("img1.jpg"); // belakang
  tex[2] = loadImage("img2.jpg"); // kanan
  tex[3] = loadImage("img3.jpg"); // kiri
  tex[4] = loadImage("img4.jpg"); // atas
  tex[5] = loadImage("img5.jpg"); // bawah

  // jika salah satu gambar gagal dimuat, buat fallback sederhana
  for (int i = 0; i < tex.length; i++) {
    if (tex[i] == null) {
      tex[i] = createImage(256, 256, RGB);
      tex[i].loadPixels();
      for (int p = 0; p < tex[i].pixels.length; p++) tex[i].pixels[p] =
color(150, 150, 150);
      tex[i].updatePixels();
      println("Warning: texture " + i + " missing, created placeholder.");
    }
  }

  textureMode(NORMAL); // koordinat (u,v) 0..1
  noStroke();
}

void draw() {
  background(25);
  //lights();
  // pencahayaan sederhana
  ambientLight(60, 60, 60);
  directionalLight(220, 220, 220, -0.5, -1, -0.8);

  // pusatkan dan atur kamera sederhana
  pushMatrix();
  translate(width/2, height/2, 0);
  rotateX(rotX);
  rotateY(rotY);

  // ukuran sisi kubus
  float s = 100;

  // DEPAN (tex[0])
  beginShape(QUADS);
  texture(tex[0]);
  vertex(-s, -s, s, 0, 0);
  vertex(s, -s, s, 1, 0);
  vertex(s, s, s, 1, 1);
  vertex(-s, s, s, 0, 1);
  endShape();

  // BELAKANG (tex[1])
  beginShape(QUADS);
  texture(tex[1]);
  vertex(s, -s, -s, 0, 0);
  vertex(-s, -s, -s, 1, 0);
  vertex(-s, s, -s, 1, 1);
  vertex(s, s, -s, 0, 1);
  endShape();

  // KANAN (tex[2])

```



```

beginShape(QUADS);
texture(tex[2]);
vertex( s, -s, s, 0, 0);
vertex( s, -s, -s, 1, 0);
vertex( s, s, -s, 1, 1);
vertex( s, s, s, 0, 1);
endShape();

// KIRI (tex[3])
beginShape(QUADS);
texture(tex[3]);
vertex(-s, -s, -s, 0, 0);
vertex(-s, -s, s, 1, 0);
vertex(-s, s, s, 1, 1);
vertex(-s, s, -s, 0, 1);
endShape();

// ATAS (tex[4])
beginShape(QUADS);
texture(tex[4]);
vertex(-s, -s, -s, 0, 0);
vertex( s, -s, -s, 1, 0);
vertex( s, -s, s, 1, 1);
vertex(-s, -s, s, 0, 1);
endShape();

// BAWAH (tex[5])
beginShape(QUADS);
texture(tex[5]);
vertex(-s, s, s, 0, 0);
vertex( s, s, s, 1, 0);
vertex( s, s, -s, 1, 1);
vertex(-s, s, -s, 0, 1);
endShape();

popMatrix();

// rotasi otomatis
rotX += 0.01;
rotY += 0.02;
}

void mouseDragged() {
  // rotasi manual dengan mouse
  rotY += (mouseX - pmouseX) * 0.01;
  rotX -= (mouseY - pmouseY) * 0.01;
}

```

### 3. Sphere dengan tekstur

Texture image: <https://www.solarsystemscope.com/textures/>

```

PImage tex;
float rotX = 0;
float rotY = 0;
boolean autoRotate = true;

void setup() {
  size(400, 400, P3D);
  tex = loadImage("earth_daymap.jpg"); // ganti dengan nama file tekstur kamu
  textureMode(NORMAL);
  noStroke();
}

void draw() {
  background(10);

```



```
// ===== Pencahayaan =====
lights();
ambientLight(40, 40, 40); // Cahaya lembut di seluruh area
directionalLight(255, 255, 255, -1, 0, -0.5); // Sinar utama dari kiri atas
pointLight(200, 200, 200, width/2, height/2, 200); // Titik cahaya depan
specular(255); // Warna highlight (pantulan)
shininess(20); // Intensitas kilau

// ===== Posisi dan rotasi bola =====
translate(width/2, height/2, 0);
if (autoRotate) rotY += 0.01;
rotateY(rotY);
rotateX(rotX);

float radius = 150; // jari-jari bola
int detail = 50; // semakin besar semakin halus

// gambar sphere dengan tekstur
texturedSphere(radius, detail, tex);
}

// ===== Fungsi untuk membuat sphere dengan tekstur =====
void texturedSphere(float r, int detail, PImage texture) {
  textureMode(NORMAL);
  beginShape(TRIANGLE_STRIP);
  texture(texture);

  for (int i = 0; i < detail; i++) {
    float theta1 = PI * i / detail;
    float theta2 = PI * (i + 1) / detail;

    for (int j = 0; j <= detail; j++) {
      float phi = TWO_PI * j / detail;

      // Koordinat vertex
      float x1 = r * sin(theta1) * cos(phi);
      float y1 = r * cos(theta1);
      float z1 = r * sin(theta1) * sin(phi);
      float x2 = r * sin(theta2) * cos(phi);
      float y2 = r * cos(theta2);
      float z2 = r * sin(theta2) * sin(phi);

      // Koordinat tekstur
      float u = (float)j / detail;
      float v1 = (float)i / detail;
      float v2 = (float)(i + 1) / detail;

      // Normal untuk pencahayaan
      normal(x1, y1, z1);
      vertex(x1, y1, z1, u, v1);
      normal(x2, y2, z2);
      vertex(x2, y2, z2, u, v2);
    }
  }
  endShape();
}

void mouseDragged() {
  // kontrol manual dengan mouse
  rotY += (mouseX - pmouseX) * 0.01;
  rotX -= (mouseY - pmouseY) * 0.01;
}

void keyPressed() {
  if (key == ' ') autoRotate = !autoRotate;
}

```



### 10.9 Soal Latihan

Buatlah program yang menampilkan sebuah kubus dengan texture tiap sisinya menampilkan texture nama tiap anggota. Image texture mengandung semua tampilan nama anggota kelompok. Tampilkan source code, texture, dan tampilan kubus yang nampak semua sisinya (3 tampilan @ 2 sisi)

#### Daftar Pustaka:

1. Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1995). Computer Graphics: Principles and Practice. Addison-Wesley.
2. Angel, E., & Shreiner, D. (2015). Interactive Computer Graphics: A Top-Down Approach Using OpenGL. Pearson.
3. Shirley, P., et al. (2022). Fundamentals of Computer Graphics. CRC Press.
4. OpenGL Documentation (2024): Texture Mapping Overview.
5. Wikipedia Commons (2025): Examples of Texture Mapping Techniques (images licensed under CC-BY-SA).
6. <https://fyeehannah.blogspot.com/2016/12/texturing-dan-rendering.html> diakses tanggal 2 November 2025.